4 Regression-Based Classification

Certain regression algorithms can also serve for classification purposes (and vice versa). These methods can add more explainability and tunable parameters then is available using purely classification algorithms.

4.1 Logistic Regression

Logistic Regression (also known as Logit Regression) is a binary classifier and is frequently utilized to estimate the probability that an instance belongs to a particular class (e.g., the probability that an email is spam). If the estimated probability exceeds 50%, the model predicts that the instance belongs to the positive class (labeled "1"); otherwise, it predicts that it does not (i.e., it belongs to the negative class, labeled "0"). Consequently, it functions as a binary classifier.

The estimated probability can be computed as

$$\hat{p} = h_\theta(X) = \sigma(\theta^{\mathrm{T}} \cdot X). \tag{4.4}$$

Here, $\hat{p}$ denotes the estimated probability and $\sigma(\cdot)$, is a sigmoid function (i.e., S-shaped) that yields a value between 0 and 1; termed the logistic or logit function. It is defined as illustrated in Equation 4.5 and depicted in Figure 4.1. As before, $h_\theta(X)$ represents the hypothesis that X belongs to a class given the parameters θ, and X denotes the matrix of input features plus a bias term

$$\sigma(x) = \frac{1}{1+e^{-x}}. \tag{4.5}$$

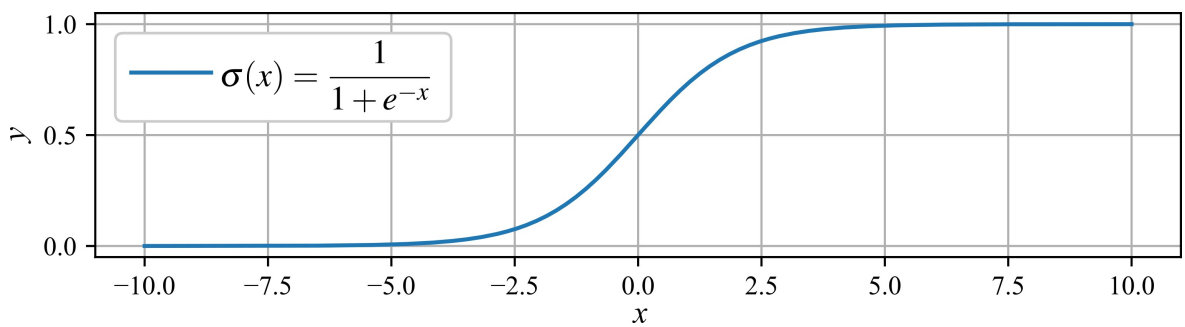

Figure 4.1: Sigmoid function that maps any real-valued input x to a value between 0 and 1.

Once the Logistic Regression model has estimated the probability $\hat{p} = h_\theta(X)$ that an instance X belongs to the positive class, it can easily make its prediction $\hat{y}$ from

$$\hat{y} = \begin{cases} 0 & \text{if } \hat{p} < 0.5, \\ 1 & \text{if } \hat{p} \geq 0.5. \end{cases} \tag{4.6}$$

Note that $\sigma(x) < 0.5$ when $x < 0$, and $\sigma(x) \geq 0.5$ when $x \geq 0$, so a Logistic Regression model predicts 1 if $\theta^{\mathrm{T}} \cdot X$ is positive, and 0 if it is negative.

Now that you comprehend how a Logistic Regression model estimates probabilities and makes predictions, let's look into an example of simplified Training and Cost Function. The objective

of training is to adjust the parameter vector θ so that the model estimates high probabilities for positive instances ($y = 1$) and low probabilities for negative instances ($y = 0$). This principle is encapsulated by the cost function shown in Equation 4.7 for a single training instance x. To achieve this, we require a cost function, as described in Equation 4.7. This can be plotted as shown in figure 4.2.

$$C(\theta) = \begin{cases} -\log(\hat{p}) & \text{if } y = 1, \\ -\log(1-\hat{p}) & \text{if } y = 0. \end{cases} \tag{4.7}$$

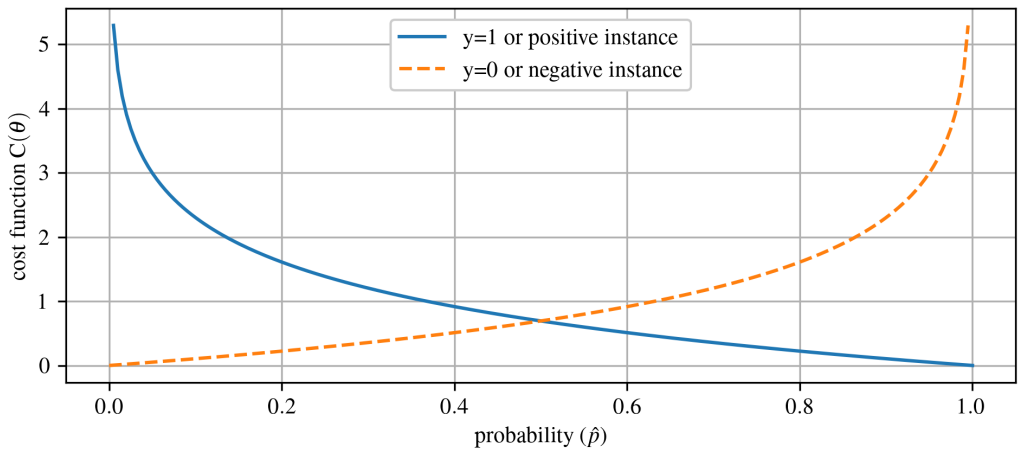

Figure 4.2: Cost function behavior for classification that heavily penalizes incorrect predictions.

The cost function shown in equation 4.7 is logical because $-\log(\hat{p})$ increases significantly as $\hat{p}$ approaches 0, resulting in a large cost if the model estimates a probability close to 0 for a positive instance. Likewise, it also yields a large cost if the model estimates a probability close to 1 for a negative instance. Conversely, $-\log(\hat{p})$ approaches 0 as $\hat{p}$ nears 1, leading to a cost close to 0 if the estimated probability is near 0 for a negative instance or near 1 for a positive instance, which aligns with our expectations.

The overall cost function, denoted as $J(\theta)$ over the entire training set, represents the average cost across all training instances. It can be expressed in a single equation, known as the log loss, depicted as

$$J(\theta) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}\log(\hat{p}^{(i)}) + (1 - y^{(i)}\log(1-\hat{p}^{(i)}))\right]. \tag{4.8}$$

While there is no closed-form equation to compute the value of θ (embedded within $\hat{p}$ in the above equation) that minimizes this cost function (unlike the Normal Equation), the cost function is convex. Therefore, Gradient Descent (or any other optimization algorithm) is assured to discover the global minimum (provided the learning rate is not excessively large and sufficient time is allowed). The partial derivatives of the cost function with respect to the jth model parameter θ_j are expressed as

$$\frac{\partial}{\partial\theta_j} = \frac{1}{m}\sum_{i=1}^{m}\left(\sigma(\theta^{\mathrm{T}}\cdot X^{(i)}) - y^{(i)}\right)x_j^{(i)}. \tag{4.9}$$

This equation bears resemblance to the equation for the partial derivative of the gradient descent. For each instance, it computes the prediction error, multiplies it by the jth feature value, and then calculates the average across all training instances. Armed with the gradient vector containing all the partial derivatives, you can employ it in the Batch Gradient Descent algorithm. That concludes the process of training a Logistic Regression model. For Stochastic Gradient Descent, you would handle one instance at a time, while for Mini-batch Gradient Descent, you would process a mini-batch at a time.

Review 4.1 Iris Flower Dataset

Let's utilize the Iris Flower dataset to demonstrate Logistic Regression. This dataset is renowned and comprises the sepal and petal length and width of 150 iris flowers belonging to three distinct species: Iris-Setosa, Iris-Versicolor, and Iris-Virginica (refer to Figure 4.3). Biologist Ronald Fisher introduced this dataset in his 1936 paper titled "The use of multiple measurements in taxonomic problems" as an illustration of linear discriminant analysis.

Figure 4.3: Flowers representing three species of iris plants [a]

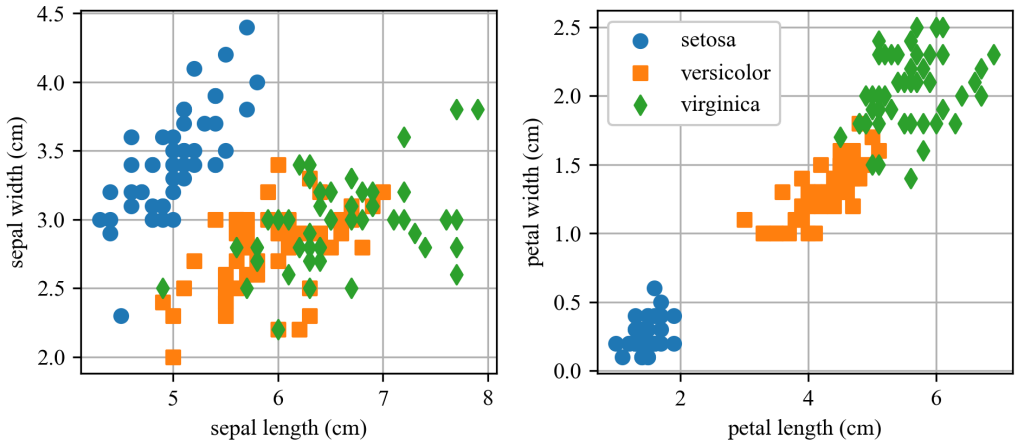

Figure 4.4: Iris dataset scatterplot showing sepal length vs. sepal width (left) and petal length vs. petal width (right).

[a]Diego Mariano, CC BY-SA 4.0 <https://creativecommons.org/licenses/by-sa/4.0>, via Wikimedia Commons

Example 4.1 Iris Dataset Exploration
This example introduces the Iris dataset (Figure 4.3), originally compiled by Ronald Fisher. It demonstrates how to load the dataset, access feature and label information, and visualize the relationships between sepal and petal measurements across the three iris species.

4.1.1 1-D Decision Boundaries

The petal width of Iris-Virginica flowers typically falls between 1.4 cm and 2.5 cm, while other iris flowers generally exhibit a narrower range, from 0.1 cm to 1.8 cm. Although there is some overlap between these ranges, the classifier's confidence varies. Beyond approximately 2 cm, the classifier is highly confident in identifying the flower as an Iris-Virginica, whereas below 1 cm, it confidently categorizes it as not an Iris-Virginica (with a high probability for the "Not Iris-Virginica" class). In the intermediate range, the classifier is uncertain. When prompted to predict the class (using the `predict()` method instead of the `predict_proba()` method), it returns the class deemed most likely. Hence, a decision boundary emerges around 1.6 cm where both probabilities reach 50%: if the petal width exceeds 1.6 cm, the classifier predicts that the flower is an Iris-Virginica; otherwise, it predicts otherwise, even if the confidence level is low.

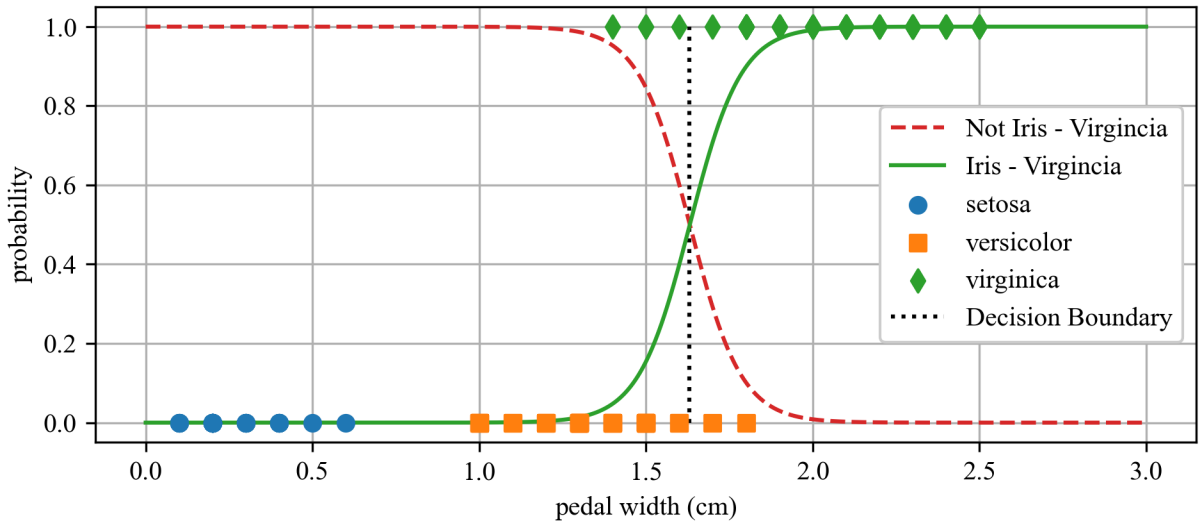

Figure 4.5: Decision boundary for the flowers of three Iris plant species with C set to $C = 10^{10}$.

Example 4.2 1D Logistic Regression
This example builds a logistic regression model to classify Iris-Virginica based on a single feature: petal width. It visualizes class probabilities and shows the decision boundary at the 50% threshold.

4.1.2 2-D Decision Boundaries

Figure 4.6 depicts the same dataset, presenting two features: petal width and length. Following training, the Logistic Regression classifier gauges the probability of a new flower being an Iris-Virginica based on these attributes. The dashed line delineates the points where the model assesses a 50% probability, indicating the decision boundary. It's worth emphasizing that this boundary is linear. Each parallel line denotes points where the model predicts specific probabilities, spanning from 10% to 90%. Flowers situated beyond the top-right line exhibit a probability exceeding 90% of being classified as Iris-Virginica by the model.

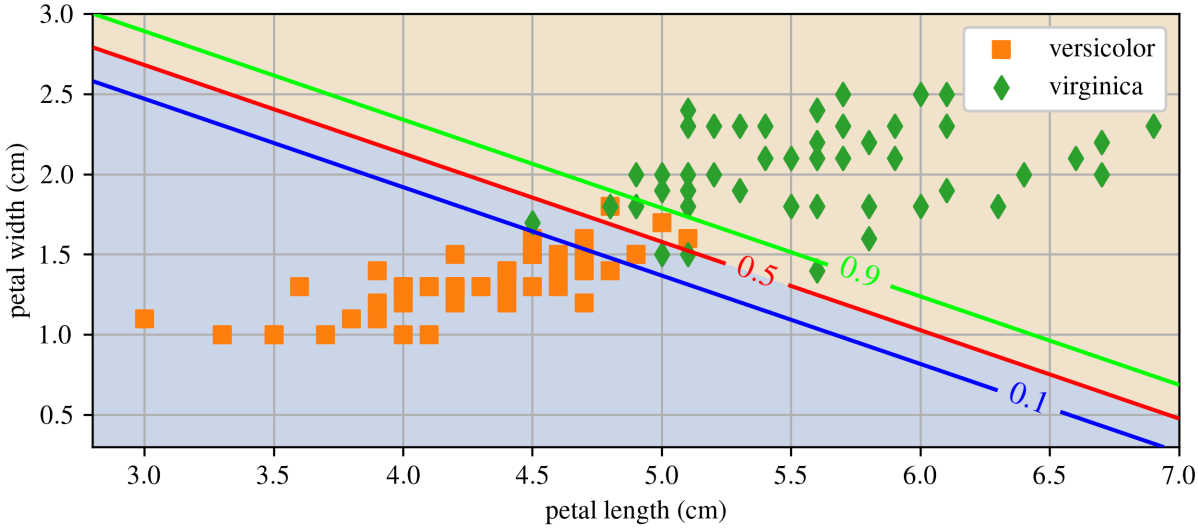

Figure 4.6: A 2D decision boundary for the Iris dataset.

Example 4.3 2D Decision Boundary
This example uses logistic regression to classify the Iris-Virginica species based on petal length and width. A 2D decision boundary is visualized in the petal feature space, with prediction regions and probability contours illustrating classifier confidence.

4.2 Softmax Regression

The Logistic Regression model can be extended to support multiple classes directly, eliminating the need to train and combine multiple binary classifiers. This is called Softmax Regression, also referred to as Multinomial Logistic Regression. Softmax Regression offers a method for multi-class regression-based classification. The principle is simple: given an instance $\boldsymbol{\Theta}$, the Softmax Regression model calculates a score $s_k(\mathbf{x})$ for each class k, and subsequently estimates the probability of each class by applying the softmax function (also known as the normalized exponential) to the scores. The formula for computing $s_k(\mathbf{x})$ may seem familiar, resembling the equation used in Linear Regression prediction

$$s_k(\mathbf{x}) = \mathbf{x}^{\mathrm{T}}\boldsymbol{\theta}^{(k)}. \tag{4.10}$$

Note that each class possesses its dedicated parameter vector $\boldsymbol{\theta}^{(k)}$, typically stored as rows in a parameter matrix Θ.

Once the scores for every class have been computed for the instance $\mathbf{x}$, the model estimates the probability $\hat{p}_k$ that the instance belongs to class k by passing the scores through the softmax function

$$\hat{p}_k = \sigma\big(s(\mathbf{x})\big)_k = \frac{\exp\big(s_k(\mathbf{x})\big)}{\sum_{j=1}^{K}\exp\big(s_j(\mathbf{x})\big)}. \tag{4.11}$$

where $\mathbf{s}(\mathbf{x})$ represents a vector containing the scores of each class for the instance $\mathbf{x}$, $\sigma\big(\mathbf{s}(\mathbf{x})\big)_k$ signifies the estimated probability that the instance $\mathbf{x}$ belongs to class k (given the scores of each

class for that instance), and K denotes the number of classes. In brief, Equation 4.11) exponentiates every score and then normalizes them (by dividing each by the sum of all exponentials).

Similar to the Logistic Regression classifier, the Softmax Regression classifier predicts the class with the highest estimated probability (which corresponds to the class with the highest score), such that

$$\hat{y} = \underset{k}{\operatorname{argmax}}\, \sigma\big(\mathbf{s}(\mathbf{x})\big)_k = \underset{k}{\operatorname{argmax}}\, s_k(\mathbf{x}) = \underset{k}{\operatorname{argmax}}\, \big((\boldsymbol{\theta}^{(k)})^{\mathrm{T}} \cdot \mathbf{x}\big). \tag{4.12}$$

NOTE

The argmax operator identifies the value of a variable that maximizes a function. In this context, it returns the value of k that maximizes the estimated probability $\sigma(s(x))_k$ within the equation.

Now that the estimation of probabilities and prediction process have been explained, let's look into training. The objective is to develop a model that assigns a high probability to the target class (and, consequently, a low probability to the other classes). Minimizing the cost function, depicted in Equation 4.13 and referred to as the cross entropy, is expected to achieve this goal as it penalizes the model for estimating a low probability for the target class. Cross entropy is commonly utilized to gauge how well a set of estimated class probabilities align with the target classes. The cost function is defined as

$$J(\Theta) = -\frac{1}{m}\sum_{i=1}^{m}\sum_{k=1}^{K} y_k^{(i)} \log(\hat{p}_k^{(i)}). \tag{4.13}$$

NOTE

In the equation, $y_k^{(i)}$ takes the value of 1 if the target class for the ith instance belongs to class k; otherwise, it takes the value of 0.

When considering only two classes ($K = 2$), it's important to highlight that this cost function aligns with the Logistic Regression's cost function, commonly referred to as log loss (refer to Equation 4.8).

The gradient vector of the cross-entropy cost function with respect to $\boldsymbol{\theta}(k)$ is expressed as

$$\nabla_{\boldsymbol{\theta}^{(k)}} J(\Theta) = \frac{1}{m}\sum_{i=1}^{m}(\hat{p}_k^{(i)} - y_k^{(i)} X^{(i)}). \tag{4.14}$$

This enables the computation of the gradient vector for every class, facilitating the utilization of Gradient Descent or any other optimization algorithm to determine the parameter matrix Θ that minimizes the cost function.

Applying Softmax Regression to classify iris flowers into all three classes involves setting Scikit-Learn's LogisticRegression to default to one-versus-all for more than two classes. However, you can switch to Softmax Regression by configuring the `multi_class` hyperparameter to "multinomial". Additionally, selecting a solver supporting Softmax Regression, such as the "lbfgs" solver, is essential (refer to Scikit-Learn's documentation for more details). By default, it incorporates $\ell 2$ regularization, adjustable using the hyperparameter C. For instance, when encountering an iris with 5 cm long and 2 cm wide petals, querying the model reveals a 94.2% probability of it being Iris-Virginica (class 2), or a 5.8% probability of being Iris-Versicolor.

NOTE

The Softmax Regression classifier is multiclass, not multioutput. As such, Softmax Regression can only predict one class at a time, so it works for problems where each input belongs to exactly one category-like classifying an email as spam, promotions, or updates. It can't be used for cases where multiple labels may apply, such as tagging a news article with topics like politics, economics, and technology all at once.

Figure 4.7 visually represents the resulting decision boundaries, distinguished by background colors. Notably, these boundaries are linear between any two classes. Additionally, the figure displays the probabilities for the Iris-Versicolor class, delineated by curved lines. Remarkably, the model can predict a class with an estimated probability below 50%, as observed at the intersection of all decision boundaries, where all classes have an equal estimated probability of 33%.

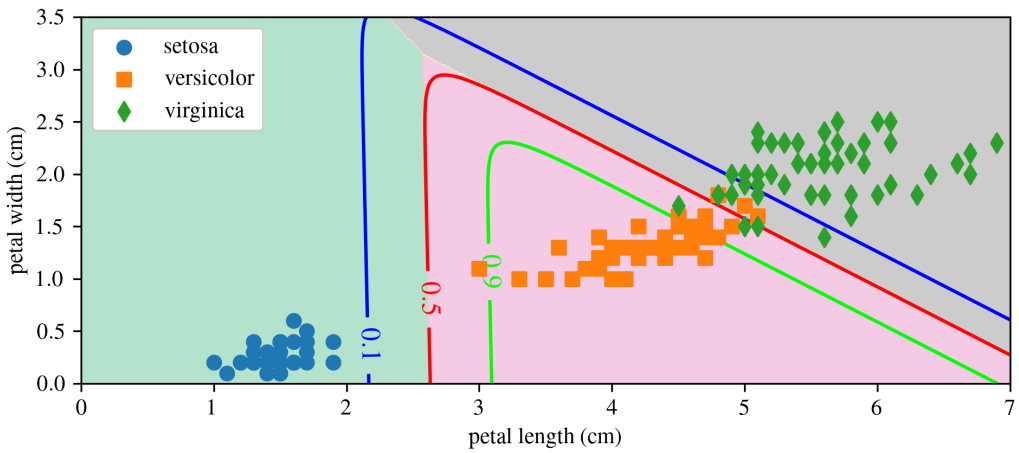

Figure 4.7: Softmax classification for the three iris plant species.

Example 4.4 Softmax Classification

This example applies softmax regression to classify all three iris species using petal length and width. It builds a multinomial logistic regression model and visualizes the decision boundaries along with confidence contours over the 2D petal feature space.